STARVNT CORE PLATFORM

DAY 5 EXECUTION PLAN

Production Integration, Reliability, Security & End-to-End Control

OWNER **Keshab - Technical Lead / Core Platform**

North Star

Prove that the StarVnt Core behaves as one secure, reliable, auditable business platform - not as a collection of disconnected modules.

REVIEW -> INTEGRATE -> TEST -> BREAK -> FIX -> RETEST -> PROVE

StarVnt Entertainment

1. Executive Intent

Day 5 is not a feature-building day. The objective is to take the authentication, business, event, automation, tenant, and intelligence boundaries established during Days 1-4 and prove that they operate together under realistic failure and security conditions.

TARGET INTEGRATION SURFACE

```
Identity
  |
Organization / Membership
  |
RBAC / Authorization
  |
Core Business
  |
Business Events
  |
Outbox -> Worker
  |
Automation -> Rules -> Conditions -> Actions
  |
Audit / Traceability / Observability
  |
Aura+ Recommendation Boundary
```

Day 5 is about	Day 5 is not about
Connecting existing components into one controlled flow	Building another CRUD module
Security and authorization proof at the API boundary	Hiding buttons in the frontend
Reliability under duplicates, races, and failures	Creating a new parallel event system
Business traceability from request to audit	Adding cosmetic logging only
Controlled Aura+ handoff into Core execution	Allowing Aura+ to mutate Core data directly

Operating principle

REVIEW -> REUSE -> EXTEND. If an existing implementation already provides the needed foundation, Keshab must review it, fix it where necessary, and extend it instead of rewriting or duplicating it.

2. Day 1-4 Architecture Audit

Before writing new code, Keshab performs a structured audit of the current platform. The purpose is to identify missing connections, duplicate sources of truth, security gaps, and reliability gaps.

Layer	Audit Questions
Authentication / Identity	Where is authentication enforced? What is the canonical user identity? What claims are trusted?
Organization / Membership	How is organization context derived? How is membership validated?
RBAC / Authorization	Where are permissions checked? Can a direct API call bypass the UI?
Core Business	Which service and collection are the source of truth for each business entity?
Events	What generates business events? What is the canonical event contract?
Outbox / Worker	Can committed business changes lose their events? Can workers safely replay work?

Automation	Where are rules, conditions, actions, execution results, and failures recorded?
Audit	Can a business change be traced to actor, resource, before, after, time, and context?
Aura+ boundary	What is the input contract? Where does recommendation end and Core authorization begin?

3. Identity -> Organization -> RBAC -> Resource Access

The first major acceptance gate is real API proof that the identity chain controls resource access.

```
LOGIN
  -> JWT
  -> USER IDENTITY
  -> ORGANIZATION CONTEXT
  -> MEMBERSHIP
  -> ROLE
  -> PERMISSION
  -> RESOURCE ACCESS
```

Scenario	Required Result
Valid user + valid JWT	Allowed according to permission
Expired JWT	Rejected
Invalid or malformed JWT	Rejected
Missing JWT	Rejected
Correct organization resource	Allowed if permission permits
Wrong organization resource	Denied
Authorized role	Allowed for permitted actions
Unauthorized role	Denied
Valid role but forbidden action	Denied
Customer attempts vendor-scoped resource access	Denied unless explicitly authorized

Critical security rule

Frontend restrictions are not authorization. Authorization must be enforced server-side, with resource ownership / tenant checks applied before a protected business operation executes.

4. Security Attack Testing

Keshab must intentionally attack the system as a hostile client. The goal is to discover whether an unauthorized request can cross the trust boundary.

Attack	Expected Outcome
Invalid JWT	401 / equivalent authentication rejection
Expired JWT	401 / equivalent authentication rejection
Modified JWT or invalid claim	Rejected
Wrong role	403 / equivalent authorization rejection
Wrong organizationId	Cross-tenant access blocked
Other organization resource ID	Blocked
Direct API request without frontend	Same authorization result as normal UI flow
Forbidden status transition	Rejected
Manipulated payload / malformed request	Validation failure; no business side effect
Duplicate request	No duplicate business action

EXPECTED TRUST-BOUNDARY BEHAVIOR

```
VALID REQUEST
  -> AUTHENTICATED
  -> AUTHORIZED
  -> BUSINESS OPERATION

INVALID / UNAUTHORIZED
  -> REJECT
  -> NO SILENT BUSINESS SIDE EFFECT
```

5. Real Business Event Pipeline

The central Day 5 engineering proof is one real business workflow that crosses the platform from a user action to an auditable automation outcome.

MASTER EVENT PATH

```
REAL USER
  |
REAL API
  |
BUSINESS DATA
  |
BUSINESS EVENT
  |
OUTBOX
  |
WORKER
  |
AUTOMATION ENGINE
  |
RULE EVALUATION
  |
CONDITION
  |
ACTION
  |
ACTIVITY / NOTIFICATION
  |
AUDIT LOG
```

Suggested real scenario

Follow-up becomes overdue -> FOLLOW_UP_OVERDUE -> automation rule matches -> reminder/activity is created -> business audit is recorded.

Outbox verification

Keshab must prove that a successfully committed business change does not silently lose its event. The business transaction and its outbox record must form a reliable boundary before the worker consumes the event.

6. Worker + Automation Verification

Check	Proof Required
Event consumption	Worker receives a real persisted event
Validation	Malformed / invalid events are rejected safely
Tenant context	Organization context survives the event path
Rule lookup	Correct automation rules are selected
Condition evaluation	Conditions are evaluated against the correct business context
Action execution	Action is executed exactly as authorized
Result persistence	Execution result is visible and traceable
Failure handling	Failed execution enters controlled retry / failed state

7. Idempotency + Concurrency

Duplicate-event protection is mandatory. A repeated business event must not create repeated business effects.

100X DUPLICATE TEST

```
FOLLOW_UP_OVERDUE

Event #1 ... Event #100
      |
      v
Idempotency detection
      |
      v
ONE effective business action
```

The same protection must survive concurrent delivery, not only sequential replay.

CONCURRENCY TEST

```
Request A -+
Request B -+
Request C -+--> same business event
Request D -+
Request E -+
          |
          v
     ONE effective action
```

Implementation proof: document the stable identity used to detect duplicates, such as organizationId + entityId + eventType + a business-specific event/condition identity. The exact mechanism must match the implemented architecture.

8. Failure Injection, Retry and Recovery

Day 5 intentionally introduces failures to validate that the system fails visibly, retries safely, and recovers without corrupting business state.

FAILURE STATE MACHINE

```
ACTION FAILS
  |
  v
```

```
LOG FAILURE
  |
  v
RETRY
  |
  +--> RETRY
  |
  v
SUCCESS

Repeated failure
  |
  v
RETRY LIMIT
  |
  v
FAILED / DEAD-LETTER STATE
  |
  v
ADMIN VISIBILITY
```

Failure Scenario	Expected Behavior
Temporary dependency failure	Retry according to defined policy
Action failure	Recorded; retried safely
Worker interruption	Unfinished work remains recoverable
Timeout	Visible failure; no silent success
Malformed event	Rejected / isolated without poisoning the worker
Repeated action failure	Retry limit reached; durable failed state visible
Dependency restored	System resumes and completes without duplicate effect

9. Business Audit Trail + Traceability

Audit must explain a business change, not merely prove that a record was updated.

MINIMUM AUDIT INFORMATION

```
WHO
WHAT
WHICH ORGANIZATION
WHICH RESOURCE
BEFORE
AFTER
WHEN
REQUEST / EVENT CONTEXT
```

Example Field	Example Value
Actor	Employee-102
Organization	ORG-001
Action	BOOKING_STATUS_CHANGED
Resource	BOOKING-501
Before	PENDING
After	CONFIRMED
Source	API / Automation
Trace	Request ID -> Event ID -> Execution ID -> Audit

Acceptance question: “Why did this business state change, through which path, and under which actor / organization context?” The platform should be able to answer this from its records.

10. Aura+ Integration Boundary

The Core must expose a controlled intelligence boundary. Aura+ consumes business context and returns structured recommendations; it does not become a second source of operational truth.

APPROVED BOUNDARY

```
CORE BUSINESS DATA
      |
      v
AURA+ INPUT BOUNDARY
      |
      v
AURA+ ANALYSIS
      |
      v
RECOMMENDATION
      |
      v
CORE HANDOFF
      |
      v
AUTHORIZATION
      |
      v
AUTOMATION
      |
      v
ACTION
      |
      v
AUDIT
```

Must be true	Must not happen
Core remains operational source of truth	Aura+ directly updates Core database records
Recommendation is structured and traceable	Aura+ bypasses Core authorization
Handoff is validated and authorized	Aura+ creates a parallel CRM / shadow business state
Execution remains auditable	Recommendation is treated as automatic authorization

11. Aura+ Recommendation -> Core Automation Handoff

CONTROLLED HANDOFF PROOF

```
AURA+
  SIGNAL: FOLLOW_UP_OVERDUE
  PRIORITY: HIGH
  RECOMMENDATION: CREATE_FOLLOW_UP_REMINDER
      |
      v
CORE
```

```
  VALIDATE ORGANIZATION
  VALIDATE ENTITY
  VALIDATE ACTION
  CHECK AUTHORIZATION
       |
       v
AUTOMATION REQUEST
       |
       v
WORKER
       |
       v
ACTIVITY
       |
       v
AUDIT
```

Important distinction: Recommendation != authorization, and recommendation != execution. Core owns the execution boundary.

12. Cross-Developer Integration Review

Owner	Keshab reviews
Nesar + Yogesh	Authentication, JWT, API contracts, roles, organization context, error handling
Sourav	Revenue CRM context, intelligence input contract, recommendation structure, organizationId, entityId, handoff contract
Keshab	Core business, automation, outbox, worker, authorization, audit, integration boundary

Contract consistency checklist

- Field naming and identifier semantics
- API request / response structure
- Error codes and validation behavior
- Status values and transitions
- Organization and entity identifiers
- Business event names
- Automation action names
- Timestamp and request-trace conventions
- Authentication claims and actor context

13. Final End-to-End Master Scenario

END-TO-END PROOF

```
USER LOGIN
  -> JWT ISSUED
  -> ORGANIZATION IDENTIFIED
  -> ROLE VERIFIED
  -> RESOURCE CREATED
  -> FOLLOW-UP CREATED
```

```
-> FOLLOW-UP BECOMES OVERDUE
-> BUSINESS EVENT
-> OUTBOX
-> WORKER
-> AUTOMATION ENGINE
-> RULE
-> CONDITION
-> AURA+ ANALYSIS
-> RECOMMENDATION
-> CORE HANDOFF
-> AUTHORIZATION
-> ACTION
-> ACTIVITY CREATED
-> AUDIT LOG
```

Then break it.

Test	Expected Proof
A. Duplicate event	Same event delivered 100x -> one effective action
B. Cross tenant	Org B user attempts Org A resource -> access denied
C. Unauthorized operation	Low-permission actor attempts protected action -> denied
D. Automation failure	Action fails -> retry -> recovery -> controlled final state

14. Day 5 Acceptance Gates

Gate	Acceptance Requirement
Authentication	Real API verified
JWT	Valid / invalid / expired tested
Organization	Context enforced
Tenant isolation	Cross-org access blocked
RBAC	Role + permission verified
API authorization	Backend enforced
Business event	Real event generated
Outbox	Event persisted reliably
Worker	Event processed
Automation	Rule executed
Conditions	Correctly evaluated
Idempotency	Duplicate protected
Concurrency	Race condition tested
Retry	Failure retries safely
Recovery	Failed action recovers
Failed / dead-letter state	Repeated failure remains visible
Audit	Business trace available
Traceability	Request -> event -> action trace
Aura+ boundary	No direct DB mutation
Handoff	Recommendation -> controlled Core execution
Security	Attack tests passed
E2E	Full business flow proven

15. Evidence Standard

“Implemented” is not sufficient. Day 5 completion must be evidenced with reproducible proof.

REQUIRED EVIDENCE FORMAT

WHAT WAS TESTED
-> HOW IT WAS TESTED
-> EXPECTED RESULT
-> ACTUAL RESULT
-> FAILURE FOUND
-> FIX APPLIED
-> FINAL RESULT

Evidence Package	Minimum Contents
Architecture Integration Report	Existing components, connected components, remaining gaps, source-of-truth decisions
Security Test Report	Attack case, expected result, actual result, status, remediation
Reliability Test Report	Duplicate, concurrency, retry, failure, and recovery evidence
E2E Proof	One complete business scenario from real request to audit
API Evidence	Real request / response examples for critical authorization and event flows
Audit Evidence	One complete business change trace
Integration Gap List	Any contract or ownership mismatch found across developers

16. Technical Leadership Standard

The question for Day 5 is not: “How many features were built?”

The question is: “Can StarVnt Core prove that it can handle real users, real business events, duplicate delivery, concurrent execution, failures, unauthorized access, and Aura+ recommendations while remaining secure, auditable, tenant-safe, and consistent?”

DAY 5 NORTH STAR

A secure, reliable, auditable Core that connects Identity -> Organization -> RBAC -> Core Business -> Events -> Automation -> Reliability -> Audit -> Aura+ through controlled execution boundaries.